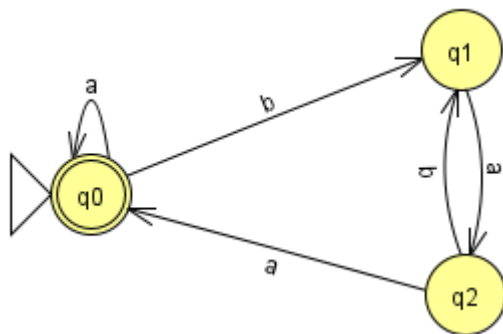


About regular expressions

A language is regular if it is accepted by some DFA, a finite deterministic automaton.
Similarly, the language $L(r)$ accepted by a regular expression r is regular.
Every regular language can be expressed by some regular expression r .

From automaton to regular expression



How to make a regular expression from this?

From the initial state to the final state

a^*

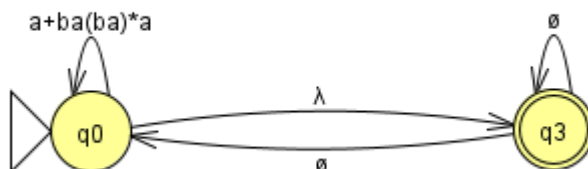
a^*baa^*

$a^* | a^*ba(baa^*b)^*aaa^*$

ta

$a^* | a^*b(aa^*b)^*aaa^*$

JFLAP:



The end result (JFLAP) is $(a+ba(ba)^*a)^*$ where $+$ stands for choice

$(a|ba(ba)^*a)^*$ as a grammar (I think this is what was asked at the lecture)

$(a|ba(ba)^*a)^*$ $S \rightarrow \lambda$ (So that it ends at some point and accepts empty)

$(a|ba(ba)^*a)^*$ $S \rightarrow aS$ (the left side of or $|$ and must return to S , because of the last $*$)

$(a|ba(ba)^*a)^*$ $S \rightarrow bB$ (the right side of or $|$)

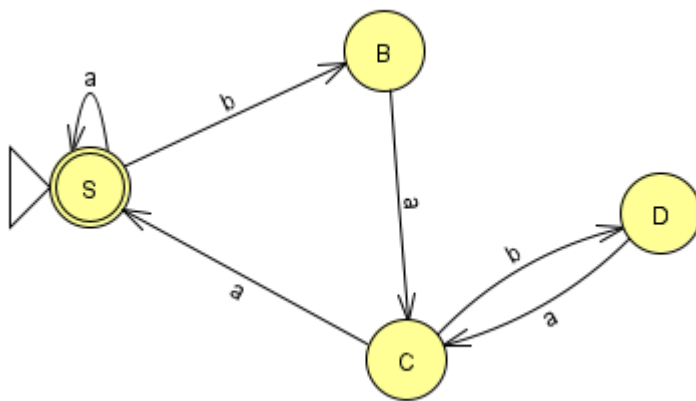
$(a|ba(ba)^*a)^*$ $B \rightarrow aC$

$(a|ba(ba)^*a)^*$ $C \rightarrow bD, D \rightarrow aC$ (ba repetition, hence $(ba)^*$)

(D can also be state B , provided that this a is the only transition out of B)

$(a|ba(ba)^*a)^*$ $C \rightarrow aS$ (We must go back to S to allow repetition, the last $*$)

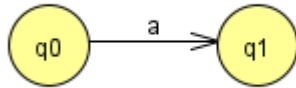
Using the transitions here, we get an automaton



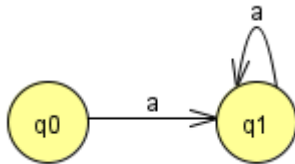
But functionality is the same as above and obviously we could combine B and D , but only because the a -transition is the same and no other outbound transitions are in B .

Regular expression to an automaton

a



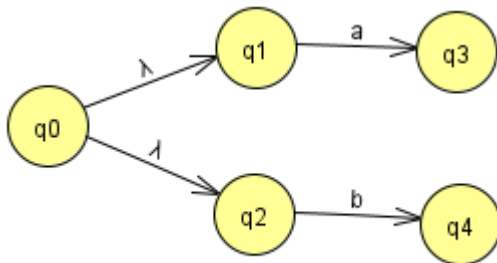
a^+



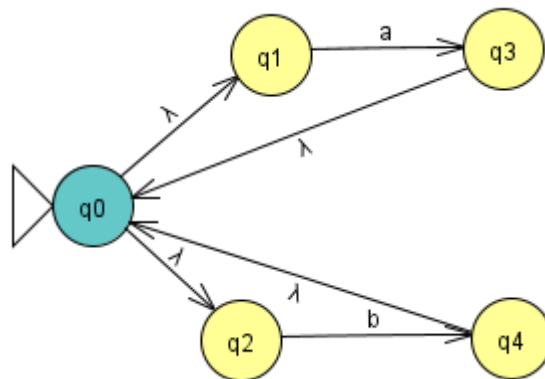
a^*



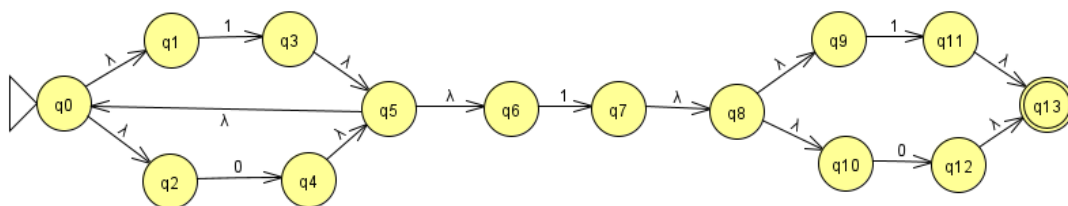
$a|b$



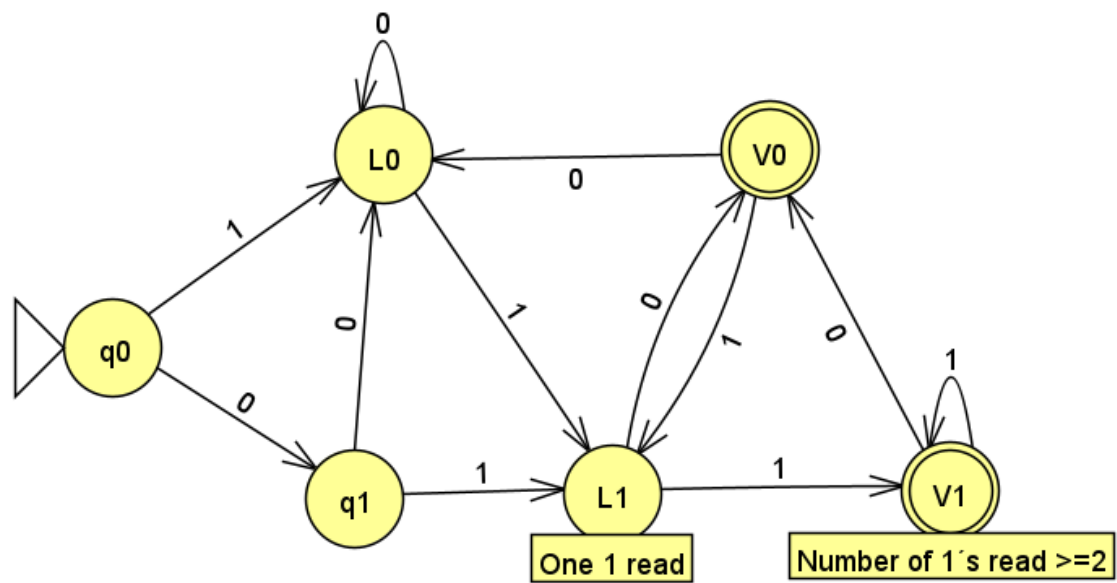
$(a|b)^*$



$(0|1)^+1(0|1) \lambda$ -NFA



The same as a DFA.



The automaton transformation by JFLAP essentially gives the same result (although with one "unnecessary" state)

